Disciplina: **Teste de Software**

Detecção de Bad Smells e Refatoração Segura

João Marcelo Carvalho Pereira Araújo

Matrícula: 857649

Repositório: github.com/joaomarcelocpa/bad-smells-js-refactoring

1. Análise de Smells (código original)

O método `generateReport` de `src/ReportGenerator.js` concentra toda a lógica do serviço. Na leitura manual identifiquei três smells principais:

1.1 Método Longo com alta complexidade (God Method)

Um único método de ~57 linhas monta cabeçalho, filtra itens por perfil, formata linhas e soma o total, tudo dentro de um `for` com `if/else if` aninhados em até quatro níveis (perfil → valor → tipo de relatório). Isso mistura quatro responsabilidades, torna o fluxo difícil de ler e faz com que qualquer novo formato ou perfil exija editar várias ramificações. Para os testes, o número de caminhos (2 tipos × 2 perfis × limites de valor) só é coberto exercitando o método inteiro, sem como testar a formatação ou a regra de acesso isoladamente.

1.2 Código Duplicado

A linha CSV (`` `${item.id},${item.name},...` ``) e a linha HTML aparecem duas vezes cada (ramos ADMIN e USER), e a instrução `total += item.value` se repete quatro vezes. Uma correção ou mudança de formato precisa ser feita em vários pontos, e esquecer um deles gera divergência silenciosa entre perfis.

1.3 Números e Strings Mágicos

Os limites `1000` (prioridade) e `500` (usuário comum) e os literais `'ADMIN'`, `'USER'`, `'CSV'`, `'HTML'` estão espalhados no código sem nome que explique a regra de negócio. Um erro de digitação em uma string falha silenciosamente (nenhum item aparece) e mudar um limite exige caçar o valor.

Outros pontos notados: o método muta o argumento (`item.priority = true`), um efeito colateral oculto; e o parâmetro `database` do construtor não é usado.

2. Relatório da Ferramenta (ESLint + eslint-plugin-sonarjs)

Executei `npx eslint src/` antes da refatoração com o `.eslintrc.json` do projeto (`sonarjs/recommended`, `cognitive-complexity` com limite 5, mais `no-identical-functions` e `no-duplicated-branches`):

```
joaomarcelocpa@MacBook-Air-de-Joao Documents % cd ~/Documents/bad-smells-print && npx eslint src/

/Users/joaomarcelocpa/Documents/bad-smells-print/src/ReportGenerator.js
  11:3   error  Refactor this function to reduce its Cognitive Complexity from 27 to the 5 allowed  sonarjs/cognitive-complexity
  43:14  error  Merge this if statement with the nested one                                          sonarjs/no-collapsible-if

✖ 2 problems (2 errors, 0 warnings)
```

Figura 1 – Saída de `npx eslint src/` antes da refatoração.

A ferramenta quantificou o que eu só tinha como impressão: a **complexidade cognitiva é 27** contra o limite de 5, um número objetivo que o SonarJS calcula penalizando cada nível de aninhamento, algo difícil de estimar de cabeça. Também apontou o `no-collapsible-if` na linha 43 (`else if (user.role === 'USER') { if (item.value <= 500) ...`), um aninhamento desnecessário que passei batido na leitura manual. Por outro lado, a ferramenta **não detectou** a duplicação nem os números mágicos (as regras de duplicação atuam sobre funções ou ramos idênticos, e aqui as linhas são parecidas, não idênticas); esses dois pontos vieram só da análise manual. As duas abordagens se complementam.

3. Processo de Refatoração

O smell mais crítico é o Método Longo / condicionais aninhadas, pois é a causa dos demais. Antes (corpo do laço, `src/ReportGenerator.js`):

```
    // --- Seção do Corpo (Alta Complexidade) ---
    for (const item of items) {
      if (user.role === 'ADMIN') {
        // Admins veem todos os itens
        if (item.value > 1000) {
          // Lógica bônus para admins
          item.priority = true;
        }

        if (reportType === 'CSV') {
          report += `${item.id},${item.name},${item.value},${user.name}\n`;
          total += item.value;
        } else if (reportType === 'HTML') {
          const style = item.priority ? 'style="font-weight:bold;"' : '';
          report += `<tr ${style}><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>
\n`;
          total += item.value;
        }
      } else if (user.role === 'USER') {
        // Users comuns só veem itens de valor baixo
        if (item.value <= 500) {
          if (reportType === 'CSV') {
            report += `${item.id},${item.name},${item.value},${user.name}\n`;
            total += item.value;
          } else if (reportType === 'HTML') {
            report += `<tr><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
            total += item.value;
          }
        }
      }
    }
```

Depois, em `src/ReportGenerator.refactored.js`, o método ficou assim, delegando para formatadores e políticas:

```
  generateReport(reportType, user, items) {
    const formatter = FORMATTERS.get(reportType);
    const policy = POLICIES.get(user.role);
    const visibleItems = policy ? selectVisibleItems(policy, items) : [];

    if (!formatter) {
      return '';
    }

    const body = visibleItems
      .map((item) => formatter.row(item, user, policy.highlightPriority))
      .join('');

    return (formatter.header(user) + body + formatter.footer(sumValues(visibleItems))).trim();
  }
}
```

Políticas e formatadores (trecho):

```
const PRIORITY_THRESHOLD = 1000;
const USER_MAX_VALUE = 500;

// --- Formatadores: um por tipo de relatório (substitui os if/else de CSV/HTML) ---
const csvFormatter = {
  header: () => 'ID,NOME,VALOR,USUARIO\n',
  row: (item, user) => `${item.id},${item.name},${item.value},${user.name}\n`,
  footer: (total) => `\nTotal,,\n${total},,\n`,
};

// --- Políticas de acesso: uma por perfil (substitui os if/else de ADMIN/USER) ---
const adminPolicy = {
  highlightPriority: true,
```

```
  prepare: (item) => {
    if (item.value > PRIORITY_THRESHOLD) {
      item.priority = true;
    }
  },
  canView: () => true,
};

const userPolicy = {
  highlightPriority: false,
  prepare: () => {},
  canView: (item) => item.value <= USER_MAX_VALUE,
};
```

Técnicas aplicadas:

- **Replace Conditional with Polymorphism:** os `if/else` por tipo de relatório viraram os objetos `csvFormatter` e `htmlFormatter` (header/row/footer), e os por perfil viraram `adminPolicy` e `userPolicy` (`prepare`/`canView`). Um novo formato ou perfil passa a ser um novo objeto registrado no `Map`, sem tocar no método principal (Open/Closed).
- **Extract Method:** `selectVisibleItems` e `sumValues` separam filtragem e soma da montagem do texto.
- **Eliminação de duplicação:** cada formato de linha existe uma só vez e o total é calculado em um único lugar.
- **Constantes nomeadas:** `PRIORITY_THRESHOLD` e `USER_MAX_VALUE`. Usei `Map` em vez de objeto literal para que valores como `"constructor"` não sejam aceitos como tipo ou perfil.

Resultado

```
$ npx eslint src/ReportGenerator.refactored.js
(sem problemas reportados; exit code: 0)
$ npm test  →  Test Suites: 2 passed, 2 total   |   Tests: 10 passed, 10 total
```

O ESLint não reporta nenhum problema no arquivo refatorado, mesmo com o limite rígido de 5 (a complexidade foi de 27 para, no máximo, 5 em cada função). Os testes originais (`ReportGenerator.test.js`, sem alteração) e a cópia apontando para o arquivo refatorado (só o `import` muda) passam 100%. Como os testes são poucos, comparei também o original e o refatorado em 6.000 combinações de tipo, perfil, valores de borda (0, 500, 501, 1000, 1001, negativos) e entradas inválidas: saídas e mutação dos itens foram idênticas.

Observação sobre a base: no repositório original o `npm test` falhava antes de qualquer alteração: (1) o projeto é ESM e o Jest exigia `--experimental-vm-modules` (ajustei o script `test`); (2) o código gerava `<tr >` com espaço quando não havia destaque, enquanto o teste espera `<tr>`. Corrigi esse espaço no original em um commit separado, para a base chegar a 100% como o enunciado exige, e o arquivo refatorado já nasce correto.

4. Conclusão

Os testes foram a rede de segurança que permitiu reestruturar todo o método com confiança: cada passo podia ser validado em segundos, e eles também mostraram, logo no início, que a base tinha um defeito que precisava ser tratado antes de refatorar. Eles têm limites, porque cobrem poucos casos, e por isso complementei com a comparação diferencial. A redução dos smells deixou o código mais legível, com responsabilidades separadas e regras de negócio nomeadas, o que facilita testar cada peça isoladamente, estender para novos formatos e perfis e reduz o risco de regressões em manutenções futuras.